SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

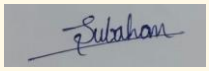
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	T. Subahan	**Reg. No.:**	192511121
Date:	27/10/26	**Duration:**	60 minutes

Code of Conduct

I T. Subahan (192511121) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.

Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.
	interfaces, and interactions accurately represented.		detail.	
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ANNEXURE A – ARCHITECTURE DESIGN ASSIGNMENT
VEHICLE SERVICE AND MAINTENANCE MANAGEMENT SYSTEM

1. INTRODUCTION

1.1 Overview of the System

The Vehicle Service and Maintenance Management System is a software application designed to simplify and automate the process of vehicle servicing and maintenance. The system provides a common platform for vehicle owners and service center staff to manage service appointments, maintain vehicle service records, receive maintenance reminders, and track the progress of vehicle servicing.

In the traditional vehicle servicing process, customers usually visit service centers directly or contact them through phone calls to book appointments. Maintaining service records manually can be difficult, and customers may forget their next scheduled maintenance date. Furthermore, vehicle owners may need to contact service center staff repeatedly to know the status of their vehicles.

The proposed system addresses these problems by providing a centralized digital platform. Vehicle owners can register their accounts, add vehicle details, select service types, book appointments, and view their service history. Service center staff can manage bookings, update service progress, record completed maintenance activities, and communicate service-related information to customers.

The system also provides automated maintenance reminders based on service schedules, previous service dates, or recorded vehicle mileage. These reminders help vehicle owners maintain their vehicles regularly and reduce the possibility of unexpected breakdowns caused by neglected maintenance.

The architecture of the system is designed using software engineering principles such as modularity, separation of concerns, abstraction, reusability, and maintainability. A Three-Tier Layered Architecture is selected to separate the presentation layer, application layer, and data layer. This separation makes the system easier to develop, test, secure, and enhance in the future.

1.2 Problem Statement

Vehicle servicing and maintenance involve several activities, including appointment scheduling, customer management, vehicle information management, maintenance record keeping, and service progress tracking. When these activities are performed manually or through disconnected systems, they can lead to appointment conflicts, missing service records, delayed communication, and inefficient service center operations.

Vehicle owners may not have convenient access to their previous service records or information about upcoming maintenance requirements. Service center employees may also face difficulties in coordinating appointments, updating service information, and maintaining accurate customer records.

Therefore, a centralized software solution is required to automate vehicle service management

and improve communication between vehicle owners and service center staff.

The proposed Vehicle Service and Maintenance Management System aims to overcome these challenges by integrating all major servicing activities into a single application. The system provides an organized, reliable, and user-friendly environment for managing the complete vehicle servicing process.

1.3 Objectives of the System

The primary objective of the system is to develop a centralized platform that simplifies vehicle servicing and maintenance management.

The specific objectives are:

1. To provide a convenient online platform for vehicle owners to book service appointments.
2. To maintain accurate digital records of vehicles and their previous service activities.
3. To reduce manual work involved in appointment scheduling and service record management.
4. To provide automated reminders for upcoming maintenance and scheduled servicing.
5. To allow vehicle owners to track the progress of their vehicles during servicing.
6. To help service center staff manage customer appointments and service requests efficiently.
7. To improve communication between vehicle owners and service center personnel.
8. To prevent appointment conflicts through availability checking and booking validation.
9. To improve the accuracy, accessibility, and security of vehicle maintenance information.
10. To design a scalable and maintainable software architecture that supports future enhancements.
11. To reduce delays and improve the overall customer experience.
12. To provide a foundation for integrating advanced features such as online payments, inventory management, and predictive maintenance.

1.4 Scope of the System

The scope of the proposed system includes vehicle owner registration, authentication, vehicle information management, appointment booking, service history management, service status tracking, and maintenance reminder generation.

Vehicle owners can use the application to manage their vehicles and access service-related information. Service center staff can use the system to view scheduled appointments, manage service requests, update service status, and record completed maintenance work.

The system maintains centralized records of customers, vehicles, appointments, and service activities. It also supports communication through email or SMS notifications using an external notification service.

The initial implementation focuses on the essential functions required for vehicle servicing. Additional features such as online payment processing, spare-parts inventory management, mechanic allocation, and AI-based maintenance prediction can be introduced in later versions.

The proposed architecture is suitable for a web-based application and can also support a mobile application through reusable APIs.

2. ANALYSIS OF SYSTEM REQUIREMENTS

2.1 System Requirements Analysis

System requirements analysis is the process of identifying the functions that the software must perform and the quality characteristics it must satisfy. These requirements influence the selection of the architecture, the division of components, the database structure, and the communication mechanisms used by the system.

For the Vehicle Service and Maintenance Management System, requirements are classified into functional requirements and non-functional requirements.

Functional requirements describe the operations provided by the application, whereas non-functional requirements describe the expected performance, security, reliability, usability, and maintainability of the system.

The architecture must support both categories to ensure that the application functions correctly and remains dependable as its usage increases.

2.2 Functional Requirements

2.2.1 User Registration and Authentication

The system shall allow new vehicle owners to create accounts by providing details such as name, email address, phone number, and password. Registered users shall be able to log in using valid credentials.

The system shall validate the login information and prevent unauthorized access. It shall also support role-based access for vehicle owners, service center staff, and administrators.

Different users shall have different permissions according to their responsibilities.

2.2.2 Vehicle Information Management

The system shall allow vehicle owners to add and manage their vehicle information. The details may include vehicle registration number, manufacturer, model, vehicle type, purchase year, and current mileage.

Each vehicle shall be associated with its registered owner. The system shall use this information to maintain service records and generate relevant maintenance reminders.

Users shall be able to view their registered vehicles and update permitted details when necessary.

2.2.3 Service Appointment Booking

The system shall provide an appointment booking facility through which vehicle owners can schedule servicing.

Users shall be able to select a vehicle, service category, preferred date, available time slot, and service center.

The application shall validate the booking request and check appointment availability before confirming the appointment. If the requested time slot is unavailable, the system shall allow the

user to select another available slot.

After successful booking, the appointment details shall be stored in the database, and a confirmation notification may be sent to the customer.

2.2.4 Appointment Management

The system shall allow authorized service center staff to view and manage scheduled appointments.

Staff shall be able to confirm appointments, update appointment details where permitted, mark cancellations, and record the arrival of vehicles.

The system shall maintain the current appointment status and ensure that booking information remains consistent.

2.2.5 Service History Management

The system shall maintain a digital record of completed vehicle servicing activities.

Each service record may contain the vehicle identifier, service date, type of service, work performed, replacement parts, mileage at servicing, service cost, and remarks provided by authorized staff.

Vehicle owners shall be able to view their previous service records. Authorized service center staff shall be able to add new service records and update them according to their permissions.

This functionality provides a complete maintenance history for each registered vehicle.

2.2.6 Service Status Tracking

The system shall allow vehicle owners to track the progress of their vehicles during servicing.

The service status may include the following stages:

1. Appointment Confirmed
2. Vehicle Received
3. Inspection in Progress
4. Service in Progress
5. Service Completed
6. Ready for Delivery
7. Vehicle Delivered

Authorized staff shall update the status as the vehicle moves through the servicing process.

The application shall display the latest recorded status to the vehicle owner. This reduces the need for repeated phone calls and improves communication between customers and service center staff.

2.2.7 Maintenance Reminder Management

The system shall generate reminders for upcoming vehicle maintenance.

Reminders may be based on the previous service date, a predefined maintenance interval, or vehicle mileage recorded by the owner or service center staff.

For example, if a vehicle requires periodic servicing after a specified time interval, the system can calculate the expected maintenance date and schedule a reminder.

The reminder component shall identify maintenance activities that are due or approaching their

due dates.

The accuracy of mileage-based reminders depends on the availability and correctness of recorded mileage information.

2.2.8 Notification Management

The system shall support notifications for important service-related events.

Notifications may include:

- Appointment confirmation.
- Appointment cancellation or rescheduling.
- Upcoming maintenance reminders.
- Changes in vehicle service status.
- Completion of vehicle servicing.
- Vehicle readiness for delivery.

The application may use an external email or SMS gateway to deliver notifications. Notification delivery failures should be recorded so that the system can retry or report the failure where appropriate.

2.2.9 Administrative Management

The administrator shall be able to manage authorized user accounts, service center information, available service categories, and system settings.

The administrator may also monitor appointment records and manage access permissions.

Administrative functions shall be protected through appropriate authorization controls.

2.2.10 Report Generation

The system may generate reports related to appointments, completed services, service costs, vehicle maintenance history, and service center activities.

These reports can help service center management understand operational performance and identify areas requiring improvement.

Access to reports shall be restricted according to the user's role.

2.3 Non-Functional Requirements

2.3.1 Scalability

The system should support an increasing number of vehicle owners, vehicles, appointments, and service centers without significant degradation in performance.

The application should allow additional application instances or stronger server resources to be introduced as usage increases.

The database should be designed with appropriate relationships, indexing, and query optimization to support growing amounts of service information.

2.3.2 Maintainability

The system should be easy to modify, test, debug, and enhance.

The application should be divided into separate modules, with each module responsible for a specific function.

Changes to appointment booking should not unnecessarily affect service history management or

authentication. Clear interfaces between components will reduce dependencies and simplify maintenance.

2.3.3 Reliability

The system should process appointment requests correctly and maintain accurate service records.

Database transactions should be used where multiple related changes must succeed together. For example, an appointment should not be confirmed if the corresponding booking operation fails.

Error handling, input validation, and regular backups should help reduce the risk of incorrect or lost information.

2.3.4 Availability

The application should remain accessible to users whenever they need to access service information or book appointments, subject to planned maintenance and infrastructure limitations.

Availability can be improved through reliable hosting, monitoring, backup procedures, and recovery mechanisms.

For a larger deployment, redundant application servers and database replication may be considered.

2.3.5 Performance

The system should respond promptly to common operations, such as login, appointment searches, booking submissions, and service history retrieval.

Efficient queries, appropriate database indexing, pagination, and optimized API operations can improve performance.

Time-consuming operations, such as sending bulk notifications, can be processed asynchronously when necessary.

2.3.6 Security

The system shall protect customer accounts, contact information, vehicle details, and service records from unauthorized access.

Passwords should be stored using secure password hashing techniques. Communication between clients and the application should use HTTPS.

Role-based access control should ensure that users can perform only the operations permitted for their roles.

The application should validate user inputs, protect sensitive information, and maintain appropriate logs of important administrative activities.

2.3.7 Usability

The application should have a simple and understandable interface.

Navigation, forms, appointment selection, service tracking, and maintenance reminders should be designed so that users can complete common tasks without unnecessary difficulty.

The interface should provide meaningful error messages and confirmation messages.

2.3.8 Data Integrity

The system should maintain accurate and consistent information across users, vehicles, appointments, and service records.

Primary keys, foreign keys, validation rules, and database transactions should be used to preserve relationships between records.

For example, a service history entry should be associated with a valid vehicle record.

2.3.9 Compatibility

The application should be accessible through commonly used web browsers and should support different screen sizes.

If a mobile application is introduced in the future, it should communicate with the same application services through documented APIs.

2.3.10 Backup and Recovery

The system should maintain regular backups of important information and provide a recovery procedure for database failures or accidental data loss.

Backup frequency and recovery targets should be defined according to business requirements.

Restoration procedures should be tested periodically to ensure that backups can actually be used during recovery.

3. SELECTION OF A SUITABLE SOFTWARE ARCHITECTURE

3.1 Introduction to Software Architecture

Software architecture defines the overall structure of a software system. It identifies the major components, their responsibilities, the relationships between them, and the methods through which they communicate.

Architecture design is an important software engineering activity because architectural decisions influence system performance, security, maintainability, reliability, and scalability.

A suitable architecture should satisfy the system's functional requirements while providing a clear structure for implementation and future development.

For the Vehicle Service and Maintenance Management System, the selected architecture must support multiple user roles, centralized data management, appointment processing, service tracking, and integration with notification services.

3.2 Available Architectural Styles

Several architectural styles can be considered for the proposed system.

3.2.1 Layered Architecture

Layered Architecture divides a software application into layers, with each layer responsible for a particular set of operations.

A common implementation contains a presentation layer, application or business logic layer, and data access layer.

Its main advantages are separation of concerns, ease of testing, and maintainability. It is suitable for business applications with clearly defined functions.

3.2.2 Client-Server Architecture

Client-Server Architecture separates the client application from the server that processes requests and manages shared resources.

In the proposed system, the vehicle owner's browser acts as a client, while the application server processes requests and retrieves information from the database.

This architecture supports centralized management and access through multiple devices.

3.2.3 Model-View-Controller Architecture

Model-View-Controller, commonly called MVC, separates an application's data model, user interface, and request-handling logic.

The Model manages data and related rules, the View displays information, and the Controller processes user actions.

MVC can be used within the presentation and application implementation of the proposed system. It is an application design pattern and can complement a layered architecture.

3.2.4 Microservices Architecture

Microservices Architecture divides an application into independently deployable services.

For example, appointment management, notification management, and service history could be implemented as separate services.

This style can support independent scaling and deployment, but it introduces additional complexity in communication, monitoring, deployment, and data consistency.

For a small or medium-sized student project, the additional infrastructure may not be necessary.

3.2.5 Event-Driven Architecture

Event-Driven Architecture allows components to communicate through events.

For example, an appointment confirmation event could trigger a notification, while a service completion event could update tracking information and initiate follow-up activities.

This approach is useful for asynchronous operations but may require a message broker, event processing mechanisms, and additional monitoring.

3.3 Selected Architecture: Three-Tier Layered Architecture

The proposed system uses a Three-Tier Layered Architecture consisting of:

1. Presentation Layer.
2. Application Layer.
3. Data Layer.

The presentation layer manages user interaction. The application layer processes business rules and coordinates operations. The data layer stores and retrieves persistent information.

The application may be deployed using a client-server model, while MVC can be used to organize its user interface and request-handling code.

The notification service is an external integration connected to the application layer.

3.4 Justification for Selecting Three-Tier Layered Architecture

3.4.1 Separation of Responsibilities

Each layer performs a clearly defined role. The presentation layer handles user interaction, the

application layer implements business rules, and the data layer manages persistence.
This separation reduces unnecessary coupling and improves the clarity of the design.

3.4.2 Ease of Development

Different developers can work on the interface, business logic, and database operations using defined interfaces.
For example, one developer can implement appointment booking screens while another develops appointment validation and database operations.

3.4.3 Improved Maintainability

The architecture allows individual modules to be modified without requiring changes throughout the entire application.
If the maintenance reminder calculation changes, the reminder module can be updated without redesigning the login interface.

3.4.4 Security and Access Control

The application layer provides a controlled point for authentication, authorization, validation, and business rule enforcement.
The client does not receive direct access to the database. Database credentials remain on the server side.

3.4.5 Centralized Data Management

The database maintains common records for users, vehicles, appointments, service history, and reminders.
This helps maintain consistency and avoids unnecessary duplication of information.

3.4.6 Support for Future Growth

The architecture can be expanded by improving the application server, deploying multiple application instances, optimizing the database, or introducing asynchronous processing.
If the system eventually becomes large and complex, selected modules can be separated into independent services.

3.4.7 Cost-Effective Implementation

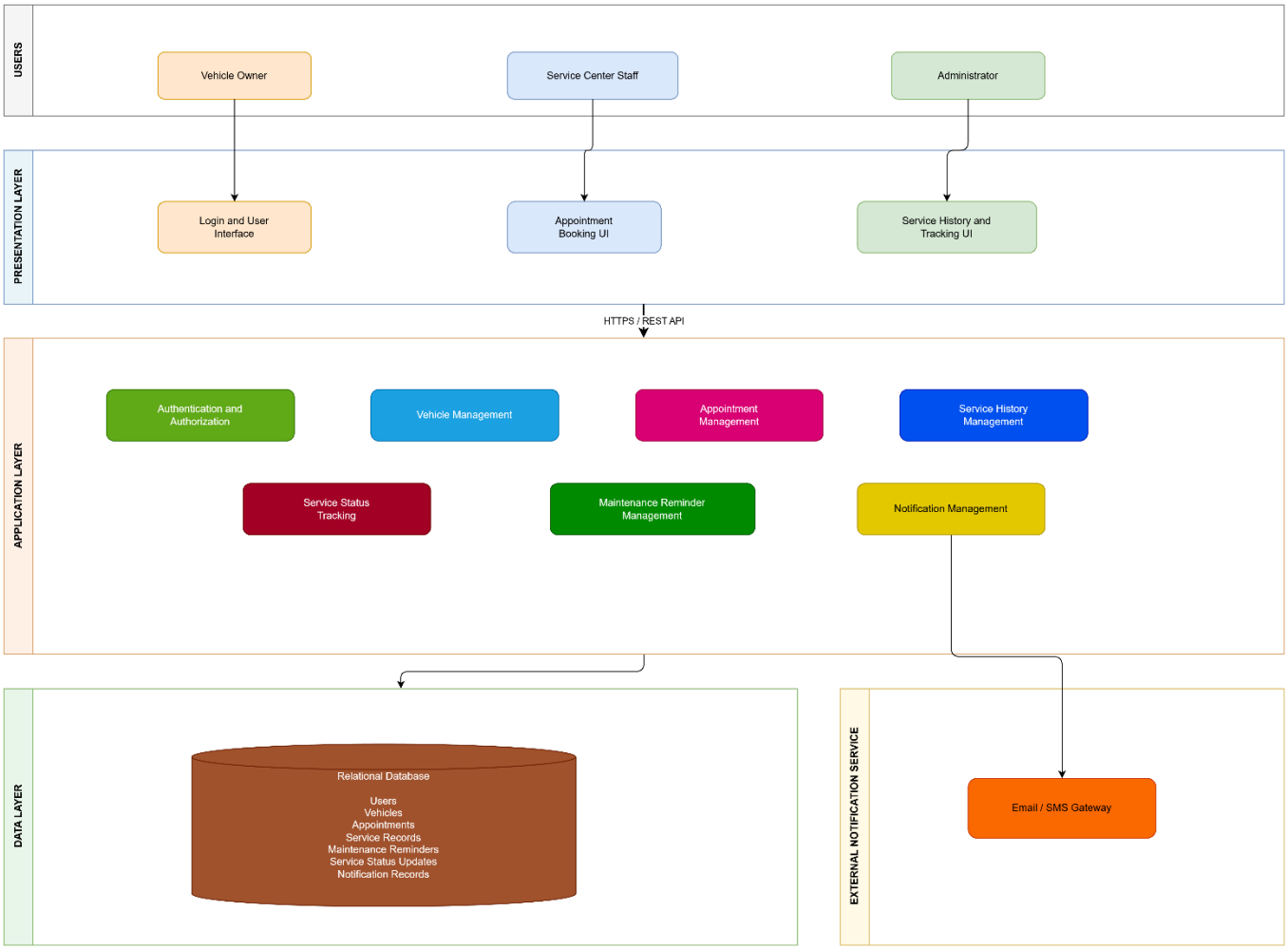
A layered application can be implemented using widely available development tools, web technologies, and relational databases.
It avoids the operational overhead of managing numerous independent services during the initial development stage.

4. SOFTWARE ARCHITECTURE DESIGN DIAGRAM

4.1 Overall Architecture

The architecture diagram illustrates the major components of the Vehicle Service and Maintenance Management System and the communication paths between them.
The diagram contains user interfaces, application services, database storage, and an external notification gateway.

Architecture Diagram:



Note: In a neatly formatted Word document, recreate this diagram using rectangular shapes and arrows. Label the three main layers and the external notification service clearly.

4.2 Explanation of the Architecture Diagram

The diagram represents the logical structure of the application.

Vehicle owners, service center staff, and administrators interact with the system through the presentation layer. The interface sends requests to the application layer using secure HTTPS communication and REST APIs.

The application layer receives and validates these requests. It checks permissions, executes business rules, and coordinates operations such as appointment booking, vehicle management, service history retrieval, and maintenance reminder generation.

When information must be stored or retrieved, the application layer communicates with the data layer. The relational database stores structured information and maintains relationships between the different entities.

When a notification is required, the application layer communicates with an external email or SMS gateway. Notification results may also be recorded in the database.

The architecture separates the user interface from the business logic and database operations,

making the application easier to maintain and enhance.

4.3 Communication Mechanisms

The system uses different communication mechanisms for different purposes.

HTTPS: HTTPS protects communication between the user interface and the application server by encrypting data in transit.

REST APIs: REST APIs allow the presentation layer to request operations such as creating appointments, retrieving service history, and checking service status.

Database Queries: The application layer uses database access components to execute authorized queries and update records.

External Service APIs: An external notification gateway may expose APIs for sending emails or SMS messages.

Scheduled Processing: A scheduler may periodically check maintenance dates and identify reminders that need to be generated.

Asynchronous Processing: A background task or message queue may be introduced to process notifications without delaying the main application request.

These mechanisms allow the components to exchange information in an organized and controlled manner.

5. EXPLANATION OF ARCHITECTURAL COMPONENTS

5.1 Presentation Layer

The presentation layer is responsible for providing the interface through which users access the application.

It displays information, collects user inputs, and presents the results returned by the application layer.

The presentation layer includes login pages, registration forms, vehicle information screens, appointment booking forms, service history pages, tracking dashboards, and maintenance reminder displays.

It should perform basic input checks to improve usability. However, important validation and security checks must also be performed on the server.

The presentation layer should not directly access the database. All operations requiring business logic or persistent data should be requested through authorized application interfaces.

5.2 Authentication and Authorization Component

This component verifies the identity of users and determines which operations they are allowed to perform.

During login, the component validates the supplied credentials using a secure authentication process.

After successful authentication, the application establishes an appropriate session or token-based access mechanism.

Authorization rules distinguish between vehicle owners, service center staff, and administrators.

For example, a vehicle owner may view their own service records, while authorized staff may update the service status of vehicles assigned to their service center.

This component reduces the risk of unauthorized access and helps protect customer information.

5.3 Vehicle Management Component

The vehicle management component maintains vehicle information and associates each vehicle with its owner.

It supports operations such as adding a vehicle, retrieving vehicle details, updating permitted information, and checking whether a vehicle exists before creating an appointment.

The component also provides the vehicle identifiers required by appointment booking, service history, and maintenance reminder modules.

Keeping vehicle management separate from other functions improves data organization and code reusability.

5.4 Appointment Management Component

The appointment management component coordinates service booking operations.

It receives appointment requests, checks the selected vehicle, validates the requested service type, verifies date and time availability, and stores confirmed appointments.

It also handles appointment cancellation and rescheduling according to the application's rules.

The component should prevent conflicting bookings. For example, two customers should not be allowed to reserve the same exclusive service slot when only one booking is permitted.

Database transactions and suitable constraints can help maintain consistency when multiple users attempt to book appointments simultaneously.

5.5 Service History Management Component

This component maintains a chronological record of vehicle servicing.

Each service record may include the service date, vehicle identifier, maintenance category, work performed, replaced parts, mileage, cost, and additional remarks.

The component retrieves previous records and stores new service information after a servicing activity is completed.

The stored information can be used to generate maintenance reminders and help customers understand their vehicle's maintenance history.

Only authorized users should be permitted to create or modify official service records.

5.6 Service Status Tracking Component

The service status tracking component maintains the current stage of each service request.

When a vehicle arrives at the service center, staff can update its status to indicate that it has been received. Further updates can represent inspection, ongoing repairs, completed work, and readiness for delivery.

The component stores status changes and makes the latest status available to authorized users.

A history of status changes can also be maintained for auditing and customer support.

This component improves transparency by allowing vehicle owners to check progress without repeatedly contacting the service center.

5.7 Maintenance Reminder Component

The maintenance reminder component identifies upcoming maintenance requirements using predefined schedules, previous service dates, and available mileage information.

A scheduled process can periodically examine service records and determine whether reminders are due.

For example, the system may generate a reminder when a vehicle approaches its configured periodic maintenance date.

The reminder rules should be configurable because maintenance intervals vary according to vehicle type, manufacturer recommendations, usage, and service requirements.

The component sends eligible reminders to the notification management component for delivery.

5.8 Notification Management Component

The notification management component coordinates the delivery of service-related messages.

It receives notification requests from other application components and prepares the appropriate message for the customer.

The message may include appointment details, service status changes, or upcoming maintenance information.

The component communicates with an external email or SMS provider through its API.

It should record notification attempts and outcomes where supported. Failed notifications can be retried according to defined policies.

Because external gateways may be temporarily unavailable, notification failure should not unnecessarily invalidate an appointment that has already been successfully stored.

5.9 Database Management Component

The database management component provides controlled access to persistent application data.

It stores information about users, vehicles, appointments, service records, maintenance reminders, and notifications.

A relational database such as MySQL or PostgreSQL is appropriate because the application contains structured information with clear relationships.

Primary keys uniquely identify records, while foreign keys help maintain relationships between related entities.

Database indexes can improve the retrieval of appointments, vehicles, and service history. Appropriate constraints and transactions help preserve data integrity.

Access to the database should be restricted to the application server and authorized administrative processes.

5.10 External Notification Service

The external notification service is responsible for delivering messages through email or SMS.

The application communicates with this service using the provider's supported API.

The gateway processes the notification request and attempts to deliver the message to the intended recipient.

The system should not assume that a successful API request always means the message has been delivered. Delivery confirmation depends on the capabilities and responses of the external provider.
The notification integration should be designed so that the provider can be changed in the future without requiring major modifications to appointment management or service tracking.

6. DATABASE DESIGN OVERVIEW

6.1 Purpose of the Database

The database provides centralized and persistent storage for the application's information.
It maintains the relationship between vehicle owners, their vehicles, service appointments, completed maintenance activities, and notifications.
A well-designed database reduces unnecessary duplication, improves consistency, and allows information to be retrieved efficiently.

6.2 Main Database Entities

6.2.1 User Table

The User table stores information about registered users.
Possible attributes include:

- User_ID – Primary key.
- Full_Name – Name of the user.
- Email – Registered email address.
- Phone_Number – Contact number.
- Password_Hash – Securely hashed password.
- Role – Vehicle owner, service staff, or administrator.
- Account_Status – Indicates whether the account is active.

The User_ID uniquely identifies each user.

6.2.2 Vehicle Table

The Vehicle table stores details of registered vehicles.
Possible attributes include:

- Vehicle_ID – Primary key.
- User_ID – Foreign key referencing the vehicle owner.
- Registration_Number – Vehicle registration identifier.
- Manufacturer – Vehicle manufacturer.
- Model – Vehicle model.
- Vehicle_Type – Type of vehicle.
- Purchase_Year – Year of purchase.
- Current_Mileage – Latest recorded mileage.

A user may own multiple vehicles, so the relationship between User and Vehicle is one-to-many.

6.2.3 Appointment Table

The Appointment table stores service booking information.
Possible attributes include:

- Appointment_ID – Primary key.
- Vehicle_ID – Foreign key referencing the vehicle.
- Service_Center_ID – Identifier of the selected service center.
- Service_Type – Requested service category.
- Appointment_Date – Scheduled date.
- Appointment_Time – Scheduled time.
- Appointment_Status – Current booking status.
- Created_At – Date and time when the booking was created.

Each appointment is associated with a specific vehicle and service center.

6.2.4 Service Record Table

The Service_Record table stores completed maintenance activities.
Possible attributes include:

- Service_Record_ID – Primary key.
- Vehicle_ID – Foreign key referencing the vehicle.
- Appointment_ID – Related appointment, where applicable.
- Service_Date – Date of service.
- Work_Performed – Description of maintenance activities.
- Parts_Replaced – Details of replaced components.
- Mileage_At_Service – Vehicle mileage at servicing.
- Service_Cost – Total recorded cost.
- Staff_Remarks – Additional service information.

A vehicle may have multiple service records over time.

6.2.5 Maintenance Reminder Table

The Maintenance_Reminder table stores scheduled maintenance notifications.
Possible attributes include:

- Reminder_ID – Primary key.
- Vehicle_ID – Foreign key referencing the vehicle.
- Reminder_Type – Type of maintenance.
- Due_Date – Expected maintenance date.
- Due_Mileage – Mileage threshold, where applicable.
- Reminder_Status – Pending, sent, completed, or cancelled.
- Notification_Date – Date when the reminder was sent.

A vehicle may have multiple reminders.

6.2.6 Service Status Table

The Service_Status table stores service progress information.
Possible attributes include:

- Status_ID – Primary key.

- Appointment_ID – Related appointment.
- Current_Status – Latest service stage.
- Updated_By – Authorized staff member.
- Updated_At – Date and time of the update.
- Remarks – Additional status information.

The system may maintain multiple status history entries for a single appointment.

6.2.7 Notification Table

The Notification table stores information about messages generated by the application.

Possible attributes include:

- Notification_ID – Primary key.
- User_ID – Intended recipient.
- Notification_Type – Email, SMS, or other supported type.
- Message_Content – Notification details or an appropriate reference.
- Created_At – Notification creation time.
- Delivery_Status – Pending, sent, failed, or confirmed delivered where supported.
- Retry_Count – Number of delivery attempts.

This table supports monitoring and troubleshooting notification operations.

6.3 Relationships Between Database Entities

The principal relationships are:

1. One user can own multiple vehicles.
2. One vehicle can have multiple service appointments.
3. One vehicle can have multiple service records.
4. One appointment can have multiple status history entries.
5. One vehicle can have multiple maintenance reminders.
6. One user can receive multiple notifications.
7. A service center can manage multiple appointments.

These relationships can be implemented using primary keys, foreign keys, and appropriate database constraints.

The database should also enforce suitable validation rules to prevent invalid references and inconsistent records.

7. OVERALL SYSTEM WORKFLOW

7.1 User Registration and Login Workflow

The workflow begins when a vehicle owner opens the application.

The user enters registration details and submits the registration form. The application validates the information, checks whether the account already exists, and stores the account details securely.

When the user logs in, the authentication component validates the credentials and establishes an authenticated session.

The user can then access the functions permitted for their role.

7.2 Vehicle Registration Workflow

After logging in, the vehicle owner opens the vehicle management section.

The owner enters the vehicle registration number, model, manufacturer, and other required details.

The application validates the submitted information and stores the vehicle record in the database.

The registered vehicle becomes available for appointment booking and maintenance history management.

7.3 Appointment Booking Workflow

The vehicle owner selects a registered vehicle and chooses the required service type.

The owner then selects a preferred service center, date, and available time slot.

The application checks the validity of the request and verifies slot availability.

If the slot is available, the appointment is created and stored in the database. The application returns a confirmation to the user.

If the slot is unavailable, the application displays an appropriate message and asks the user to select another time.

A notification request may then be submitted to the notification component.

7.4 Vehicle Servicing Workflow

When the vehicle arrives at the service center, authorized staff retrieve its appointment details.

The staff update the service status to indicate that the vehicle has been received.

After inspection, staff update the status to show that servicing or repairs are in progress.

When the work is completed, the staff record the maintenance activities, mileage, parts replaced, and service cost.

The service status is updated to indicate completion and readiness for delivery.

The system stores the service record and makes the relevant information available to the vehicle owner.

7.5 Maintenance Reminder Workflow

A scheduled process checks vehicle service information and configured maintenance intervals.

The system identifies vehicles approaching their next maintenance date or mileage threshold.

For each eligible vehicle, the reminder component creates a reminder and submits a notification request.

The notification component sends the reminder through the configured external gateway.

The system records the notification status and avoids repeatedly sending the same reminder unnecessarily.

7.6 Service Status Tracking Workflow

The vehicle owner logs in and opens the service tracking section.

The application retrieves the relevant appointment and its latest status from the database.

The status is displayed through the user interface.

Whenever authorized staff update the status, the latest information becomes available to the customer. A notification may also be sent if the status change requires customer communication.

7.7 Workflow Summary

The complete workflow can be summarized as follows:

1. User registers and logs in.
2. User adds vehicle information.
3. User selects a service and books an appointment.
4. The application validates and stores the appointment.
5. The user receives confirmation.
6. Service center staff receive the vehicle.
7. Staff update the service progress.
8. Completed maintenance details are recorded.
9. The user views the updated status and service history.
10. The system schedules and sends future maintenance reminders.

This workflow demonstrates how the different components cooperate to support the complete servicing process.

8. EVALUATION OF QUALITY ATTRIBUTES

Quality attributes describe the characteristics that determine how effectively a software system operates under expected conditions.

The proposed architecture is evaluated against scalability, security, performance, maintainability, and availability.

8.1 Scalability

Scalability is the ability of a system to accommodate an increase in users, requests, and stored data without unacceptable performance degradation.

The Vehicle Service and Maintenance Management System may initially serve a single service center but may later expand to support multiple service centers and a larger customer base.

The selected architecture supports scalability by separating the user interface, application logic, and database.

Architectural Support for Scalability

Application Server Scaling: Additional application instances can be deployed to process requests from more users.

Load Balancing: A load balancer can distribute requests across multiple application instances.

Database Optimization: Indexing, query optimization, and connection management can improve the handling of growing amounts of data.

Caching: Frequently accessed information that does not require immediate database consistency may be cached where appropriate.

Asynchronous Processing: Notification generation and other background activities can be

processed outside the main request-response path.

Evaluation

The layered architecture provides a practical foundation for horizontal or vertical scaling.

However, the database and shared application resources may become bottlenecks if the system grows significantly. These areas should be monitored and optimized as usage increases.

8.2 Security

Security ensures that application information and operations are protected against unauthorized access, modification, and disclosure.

The system stores personal contact details, vehicle information, and service history. Therefore, appropriate security controls are required.

Architectural Support for Security

Authentication: The system verifies user identities before providing access to protected functions.

Authorization: Role-based access control restricts operations according to user responsibilities.

Secure Communication: HTTPS protects data transmitted between the client and application server.

Password Protection: Passwords are stored using secure password hashing rather than plain text.

Input Validation: Server-side validation reduces the risk of invalid input and common application attacks.

Database Protection: Database credentials and access permissions are restricted to authorized application processes.

Audit Logging: Important administrative operations and service record changes can be logged for accountability.

Backup Protection: Backups should be protected through appropriate access controls and encryption where required.

Evaluation

The architecture centralizes important security controls in the application layer.

However, the layered structure alone does not guarantee security. Correct implementation, secure configuration, dependency updates, monitoring, and periodic security testing are also necessary.

8.3 Performance

Performance refers to how quickly and efficiently the system processes requests.

The application should respond efficiently when users log in, book appointments, retrieve service history, and track service progress.

Architectural Support for Performance

Efficient Database Queries: Queries should retrieve only the required information.

Database Indexing: Indexes on frequently searched fields can improve record retrieval.

Pagination: Large service histories and appointment lists should be retrieved in manageable

sections.

Caching: Frequently requested data may be cached when appropriate.

Asynchronous Notifications: Sending email and SMS notifications can be performed in the background to avoid delaying the main operation.

Resource Management: Database connections and application resources should be managed efficiently.

Evaluation

The architecture separates the main business operations from data storage, allowing each area to be optimized independently.

Performance testing should be conducted using realistic workloads to identify bottlenecks and establish measurable response-time targets.

8.4 Maintainability

Maintainability refers to the ease with which software can be corrected, improved, and adapted to changing requirements.

The system may require future modifications, such as adding a new service category, changing maintenance rules, or integrating an online payment gateway.

Architectural Support for Maintainability

Modular Design: Each major function is implemented in a separate module.

Separation of Concerns: User interface logic, business rules, and database operations are separated.

Reusable Components: Common functions, such as authentication and notification generation, can be reused.

Testing: Individual modules can be unit tested, while API and integration tests verify communication between components.

Documentation: Clear documentation helps developers understand component responsibilities and dependencies.

Version Control: Source code changes can be tracked using version control tools.

Evaluation

The architecture reduces unnecessary coupling between components and makes routine changes easier.

However, maintainability depends on good coding practices, consistent interfaces, appropriate testing, and proper documentation.

8.5 Availability

Availability refers to the ability of the application to remain accessible and operational when required.

The system should allow users to retrieve service information and make bookings without unnecessary interruptions.

Architectural Support for Availability

Reliable Hosting: The application should run on appropriately maintained infrastructure.

Health Monitoring: Monitoring tools can identify failures and performance problems.
Database Backups: Regular backups support recovery from data loss.
Recovery Procedures: Documented procedures should define how services are restored after failures.
Redundant Application Instances: Multiple instances can reduce the effect of an individual application server failure.
Database Recovery: Appropriate replication or restoration strategies can reduce downtime and data loss.
Error Handling: The application should handle failures gracefully and provide useful messages without exposing sensitive technical information.

Evaluation

The proposed architecture can support good availability when combined with reliable infrastructure and appropriate operational procedures.

A basic single-server implementation may still contain single points of failure. Higher availability requires additional infrastructure, testing, monitoring, and recovery planning.

9. JUSTIFICATION OF ARCHITECTURAL DECISIONS

9.1 Rationale Behind the Selected Architecture

The Three-Tier Layered Architecture was selected because the system contains clearly defined functions and requires a structured method for managing user interaction, business operations, and persistent data.

The presentation layer allows different users to interact with the application. The application layer implements business rules and coordinates the different modules. The data layer stores the records required by the application.

This separation makes the system easier to understand, develop, test, and maintain.

The architecture also allows the system to use a centralized database and common application services. Vehicle owners and service center staff can access the same consistent information according to their permissions.

The selected architecture provides an appropriate balance between implementation simplicity and the ability to support future improvements.

9.2 Reasons for Not Selecting Microservices Initially

Microservices Architecture could be used to implement appointment management, service history, notifications, and other functions as independent services.

However, this approach introduces additional requirements, including inter-service communication, service discovery or routing, distributed monitoring, deployment management, and handling data consistency across services.

For a student-level project or an application serving a limited number of service centers, these additional complexities may not provide sufficient benefits.

A modular layered application is easier to implement and maintain during the initial

development stage.

If the system later expands significantly, modules with independent scaling or deployment requirements can be considered for migration to separate services.

9.3 Reasons for Selecting a Relational Database

The system contains structured information and clear relationships between users, vehicles, appointments, and service records.

For example, one user may own several vehicles, and each vehicle may have several appointments and maintenance records.

A relational database supports these relationships through primary keys, foreign keys, constraints, and transactions.

It also supports structured queries for retrieving service histories, finding available appointments, and generating operational reports.

Therefore, a relational database such as MySQL or PostgreSQL is a suitable choice for the initial implementation.

9.4 Reasons for Using REST APIs

REST APIs provide a standard interface through which the presentation layer communicates with the application layer.

The same APIs can support a web application and a future mobile application.

For example, the application may expose operations for registering vehicles, creating appointments, retrieving service history, and viewing service status.

Well-defined API contracts reduce dependencies between the user interface and business logic.

Authentication, authorization, input validation, and error handling must be applied to the API endpoints.

9.5 Reasons for Separating Notification Management

Notification delivery depends on an external service that may experience delays or temporary failures.

Separating notification management from appointment and service tracking allows the application to handle these operations independently.

For example, an appointment can be successfully stored even if an email provider is temporarily unavailable.

The system can record the failed notification and attempt delivery again according to a retry policy.

This separation improves modularity and reduces the impact of external service failures on the main business operations.

9.6 Architectural Trade-Offs

Every architecture involves trade-offs between simplicity, flexibility, cost, and operational complexity.

The major trade-offs for the proposed system are described below.

9.6.1 Simplicity Versus Independent Deployment

A layered application is simpler to deploy than a large collection of microservices.
However, individual modules may not be independently deployable.
This trade-off is acceptable for the initial version because the system does not require independent deployment of every function.

9.6.2 Centralized Database Versus Independent Data Ownership

A shared relational database simplifies queries and maintains consistent relationships between records.
However, a heavily used shared database may become a bottleneck as the system grows.
The initial design prioritizes simplicity and data consistency. Database optimization, read replicas, or service-specific data ownership may be considered in later versions.

9.6.3 Synchronous Processing Versus Asynchronous Processing

Synchronous processing is simple and suitable for operations that require an immediate result, such as checking appointment availability.
However, long-running operations can increase response time.
Asynchronous processing is more appropriate for notifications and scheduled background tasks, although it introduces additional handling for retries, failed jobs, and processing status.

9.6.4 Development Cost Versus High Availability

A single-server implementation is less expensive and easier to manage.
However, a failure of that server may make the application unavailable.
Multiple application instances and redundant infrastructure improve availability but increase deployment costs and operational complexity.
The initial design can use a simpler deployment, while larger deployments can introduce redundancy according to business requirements.

9.6.5 Security Versus Convenience

Additional authentication and authorization controls improve protection but may introduce extra steps for users.
The system should apply strong security controls while maintaining a straightforward user experience.
For example, role-based permissions can operate in the background without requiring vehicle owners to understand the internal security design.

10. FUTURE ENHANCEMENTS

The proposed architecture provides a foundation for adding new features as the needs of vehicle owners and service centers evolve.

10.1 Online Payment Integration

An online payment gateway can be integrated to allow customers to pay service charges through supported digital payment methods.
The payment component should verify transaction results and maintain appropriate payment records.

Sensitive payment information should be handled according to the payment provider's requirements.

10.2 Mobile Application Development

A mobile application can be developed for Android and iOS platforms.

It can reuse the existing REST APIs for login, appointment booking, service tracking, and maintenance reminders.

This allows users to access important information through mobile devices without duplicating the core business logic.

10.3 AI-Based Predictive Maintenance

Artificial intelligence and machine learning techniques may be introduced to estimate maintenance needs using historical service records, mileage, vehicle usage patterns, and other available data.

The system could identify patterns associated with recurring maintenance problems and provide recommendations.

The reliability of such predictions would depend on the quality and quantity of available data, as well as proper model validation.

10.4 IoT-Based Vehicle Monitoring

The system may be integrated with suitable vehicle sensors or telematics devices.

Such devices could provide information about mileage, engine condition, battery status, or other supported parameters.

The application could use this information to generate more timely maintenance alerts.

This enhancement would require secure device communication, data validation, and appropriate management of device access.

10.5 Spare-Parts Inventory Management

A spare-parts inventory module can be introduced to track available parts, stock levels, and parts used during servicing.

The module could help service center staff identify required parts and reduce delays caused by unavailable inventory.

Integration with service records would improve the accuracy of inventory usage information.

10.6 Multi-Service-Center Support

The system can be expanded to support multiple service centers.

Each service center could manage its own staff, appointment slots, services, and maintenance records according to defined access permissions.

The application would need to enforce appropriate separation of service center data.

10.7 Advanced Reporting and Analytics

Reporting functions can be enhanced to display appointment trends, common maintenance activities, service completion times, and service center workload.

Managers could use dashboards to understand operational performance and make informed decisions.

10.8 Cloud Deployment

The application can be deployed on a cloud platform to support flexible resource allocation and centralized access.

Cloud hosting may provide managed databases, monitoring, backup facilities, and scalable application resources.

The selected services and configurations should be based on cost, security, availability, and performance requirements.

10.9 Real-Time Service Updates

The application can introduce real-time updates using WebSockets or another suitable communication mechanism.

Users could receive immediate status changes without manually refreshing the page.

This enhancement would be useful when service centers need to provide frequent progress updates.

10.10 Integration with External Vehicle Services

The system may integrate with authorized vehicle information services, manufacturer maintenance schedules, or other service center applications where suitable APIs are available.

Such integrations can reduce duplicate data entry and improve the usefulness of maintenance information.

11. IMPLEMENTATION AND TESTING CONSIDERATIONS

11.1 Suggested Technology Stack

The following technologies can be considered for implementing the proposed architecture.

Presentation Layer: HTML, CSS, JavaScript, and a frontend framework such as React.

Application Layer: Java with Spring Boot, Python with Django or FastAPI, or another suitable server-side framework.

Data Layer: MySQL or PostgreSQL.

Communication: HTTPS and REST APIs.

Notification Integration: An email service or SMS gateway that provides an appropriate API.

Version Control: Git.

Testing Tools: Unit testing frameworks, API testing tools, and browser-based testing tools.

These are suggested technologies rather than mandatory requirements. The final selection should depend on the project's scope, team experience, and available development environment.

11.2 Unit Testing

Unit testing verifies that individual components operate correctly.

For example, appointment validation can be tested with valid dates, invalid dates, unavailable time slots, and conflicting booking requests.

The maintenance reminder module can be tested with different maintenance intervals and due dates.

Authentication and authorization rules should also be tested independently.

11.3 Integration Testing

Integration testing verifies that different components communicate correctly.

Examples include:

- Testing the interaction between appointment management and the database.
- Testing the relationship between service records and vehicle information.
- Testing the interaction between service status tracking and notification management.
- Testing the communication between the application and an external notification provider.

These tests help identify problems that may not appear when components are tested individually.

11.4 System Testing

System testing evaluates the complete application against its requirements.

A typical test scenario may involve registering a user, adding a vehicle, booking an appointment, updating service status, recording completed service work, and retrieving the resulting service history.

The system should be tested under normal conditions and appropriate failure scenarios.

11.5 Security Testing

Security testing checks whether unauthorized users can access restricted functions or information.

It should include tests for role-based permissions, invalid authentication attempts, input validation, session management, and secure API access.

Sensitive information should not be exposed through error messages or application logs.

11.6 Performance Testing

Performance testing evaluates response times and system behavior under different workloads.

The application can be tested with increasing numbers of users making appointment requests and retrieving service history.

The results can be used to identify bottlenecks and determine whether additional resources or optimization are required.

11.7 Recovery Testing

Recovery testing verifies whether the application can restore its data and resume operations after a failure.

Database restoration procedures should be tested using appropriate backups.

Recovery objectives should be established according to the importance of the data and the service requirements.

12. EXPECTED BENEFITS OF THE PROPOSED ARCHITECTURE

The proposed architecture provides several benefits to vehicle owners, service center staff, and administrators.

12.1 Benefits to Vehicle Owners

Vehicle owners can book appointments conveniently without needing to visit the service center

just to schedule a service.
They can access their service history, view service progress, and receive maintenance reminders through a common application.
This improves convenience and helps users maintain their vehicles more regularly.

12.2 Benefits to Service Center Staff

Service center staff can manage appointments and service records through a centralized system.
The application reduces dependence on manual records and simplifies the process of updating service progress.
Access to appointment and vehicle information also supports better coordination of daily service activities.

12.3 Benefits to Administrators

Administrators can manage user permissions, service categories, and system settings.
Centralized records and reporting functions provide better visibility into system usage and service operations.

12.4 Benefits to Software Developers

The layered structure provides a clear organization for application development.
Developers can work on individual modules, test components independently, and introduce changes with reduced impact on unrelated functions.
The architecture also provides a foundation for integrating additional services in the future.

12.5 Benefits to the Organization

The proposed system can reduce manual administrative work, improve record accuracy, and provide a more organized customer service process.
Although the actual benefits depend on implementation quality and adoption, the architecture provides a structured foundation for digital vehicle service management.

13. CONCLUSION

The Vehicle Service and Maintenance Management System is designed to simplify vehicle servicing by providing appointment booking, vehicle information management, digital service history, service status tracking, and automated maintenance reminders.
The architecture design process begins with the identification of system objectives and the analysis of functional and non-functional requirements. These requirements guide the selection of an architecture that can support the application's operational needs and quality attributes.
A Three-Tier Layered Architecture has been selected because it separates the presentation layer, application layer, and data layer. This separation provides a clear division of responsibilities, improves maintainability, supports centralized data management, and creates opportunities for future expansion.
The architecture also supports integration with external notification services through defined APIs. Appropriate implementation practices, including authentication, authorization, input validation, database transactions, monitoring, backups, and testing, are required to achieve the

expected quality attributes.

Although a layered architecture may require additional infrastructure and optimization as the application grows, it provides a practical and cost-effective foundation for the initial implementation.

Future enhancements may include online payments, mobile applications, AI-based predictive maintenance, IoT-based vehicle monitoring, inventory management, cloud deployment, and real-time service updates.

In conclusion, the proposed architecture provides a structured, maintainable, secure, and extensible foundation for developing the Vehicle Service and Maintenance Management System. It demonstrates how software engineering principles can be applied to design a practical solution for improving vehicle service operations and customer experience.